

作业 2 Unix V6++内存分配，地址映射与内存保护

同济大学计算机系操作系统 国豪 2023 拔尖班 2023
姓名 毛星博 学号 2351520

一、内存分配 & 为进程建立地址映射关系



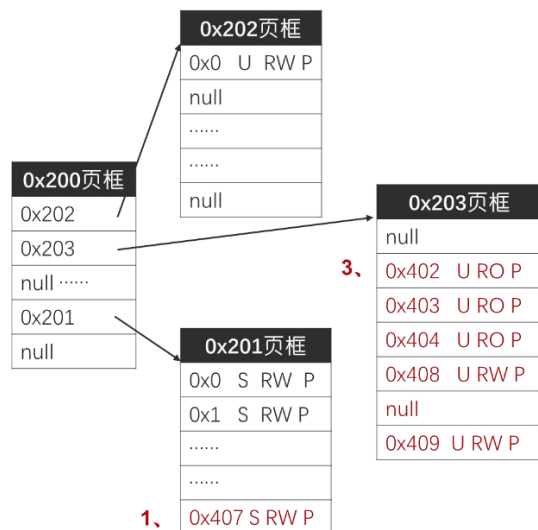
T0 时刻，用户区如图所示（注释：阴影部分已分配，除了 PA 正文段和可交换部分，还紧贴存放有其它进程的正文段和可交换部分）。PA 进程代码段 3 页，数据段 1 页，堆栈段 1 页；x_caddr=0x402000，p_addr=0x407000。PA 创建子进程 PB。回答以下问题

- 1、PB 进程， x_caddr = (0x402000) ， p_addr = (0x410000)
- 2、画 PA 的相对表，PB 的相对表

		PA的相对表		PB的相对表	
正文段	0	null		null	
	1	null		null	
					
	代码，第0页	1024	null		null
代码，第1页	1025	0	U RO P	0	U RO P
代码，第2页	1026	1	U RO P	1	U RO P
可交换部分	1027	2	U RO P	2	U RO P
	1028	1	U RW P	1	U RW P
					
		null		null	
	2047	2	U RW P	2	U RW P

图 1

- 3、调度 PA，执行前为其建立地址映射关系



PA进程代码段3页，数据段1页，堆栈段1页；
x_caddr=0x402000, p_addr=0x407000

1、SwthUStruct(selected);

p_addr >> 12, 填入核心页表1023#PTE的 base 字段, 建立PPDA区的地址映射关系。
#define SwthUStruct(p) \ Machine::Instance().GetKernelPageTable().m_Entry[Kernel::USER_PAGE_INDEX].m_PageBaseAddress \ = (p)->p_addr / PageManager::PAGE_SIZE; \ FlushPageDirectory();

2、User& newu = Kernel::Instance().GetUser();
newu 是装在物理页框0x407里, PA进程的user结构

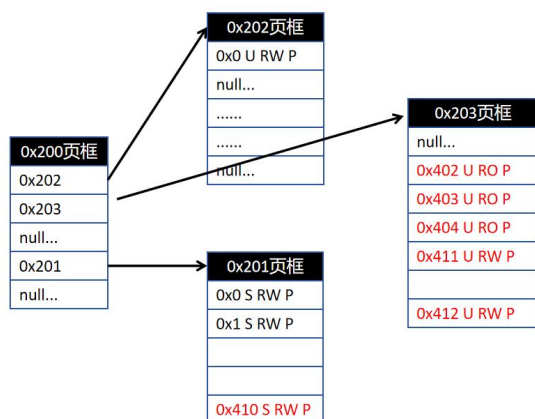
3、newu.u_MemoryDescriptor.MapToPageTable();
以PA的相对表为模板, 写0x202,0x203#物理页框

图 2

4、调度 PB, 执行前为其建立地址映射关系

请阅读源代码, 画系统页表

PB 页表如下图所示:



PB 进程代码段3页，数据段1页，堆栈段1页；
x_caddr=0x402000, p_addr=0x410000

1、SwthUStruct(selected);

p_addr >> 12, 填入核心页表1023#PTE的 base 字段, 建立PPDA区的地址映射关系。

2、User& newu = Kernel::Instance().GetUser();
newu 是装在物理页框 0x410 里, PB 进程的user结构。

3、newu.u_MemoryDescriptor.MapToPageTable();
以 PB 的相对表为模板, 写0x202,0x203#物理页框。

5、进程切换是指 CPU 换现运行进程。上述第 3、4 小题展示进程切换时, 为新选中进程建立地址映射关系。请修改 MapToUserPageTable() 函数, 尝试不使用相对表, 用 (1) 进程内存描述符中的其它字段 (2) x_caddr (3) p_addr 为新选中进程建立地址映射关系。写设计思路和伪代码。

设计思路:

1.准备阶段:

和原函数一样, 首先获取指向系统真实用户页表 (位于 0x202, 0x203) 的指针 pUserPageTable。

将整个用户页表区清零, 确保没有上一个进程的残留映射。

计算出代码段的物理起始页框号 textPF = x_caddr >> 12 和可交换部分的物理起始页框号 pAddrPF = p_addr >> 12。

2.映射代码段 (Text Segment):

根据 `m_TextSize` 计算出代码段需要多少个页, $\text{numTextPages} = \text{m_TextSize} / \text{PAGE_SIZE}$ 。

从 `m_TextStartAddress` 开始, 进行一个 `numTextPages` 次的循环。

在每次循环 `i` (从 0 到 `numTextPages - 1`) 中:

计算出当前代码页的虚拟页号 (VPN)。

计算出当前代码页的物理页框号 (PFN), 公式为 $\text{textPF} + i$ 。

在 `pUserPageTable` 中找到对应 VPN 的位置, 填入 PFN 和保护属性 (U RO P)。

3.映射数据段 (Data Segment):

根据 `m_DataSize` 计算出数据段的页数 `numDataPages`。

从 `m_DataStartAddress` 开始, 进行一个 `numDataPages` 次的循环。

在每次循环 `i` 中:

计算出当前数据页的虚拟页号 (VPN)。

计算出当前数据页的物理页框号 (PFN)。这里需要注意, 可交换部分的物理布局是 PPDA -> 数据 -> 堆栈。因此数据段的物理页框号要跳过 PPDA 所占的 1 页, 公式为 $\text{pAddrPF} + 1 + i$ 。

在 `pUserPageTable` 中找到对应 VPN 的位置, 填入 PFN 和保护属性 (U RW P)。

4.映射用户栈 (User Stack):

根据 `m_StackSize` 计算出堆栈的页数 `numStackPages`。

堆栈的虚拟地址通常位于用户空间的顶部。我们需要计算出栈顶的虚拟页号。

进行 `numStackPages` 次循环。

在每次循环 `i` 中:

计算出当前堆栈页的虚拟页号 (VPN)。

计算出当前堆栈页的物理页框号 (PFN)。它在物理上紧随数据段之后, 公式为 $\text{pAddrPF} + 1$ (for PPDA) + `numDataPages` + `i`。

在 `pUserPageTable` 中找到对应 VPN 的位置, 填入 PFN 和保护属性 (U RW P)。

5.收尾阶段:

调用 `FlushPageDirectory()` 刷新 MMU 的 TLB 缓存, 使新的映射生效。

代码:

// `USER_VIRTUAL_TOP_ADDRESS` 是用户虚拟地址空间顶部的地址

```
void Modified_MapToPageTable()
```

```
{
    // 1. 准备阶段
    User& u = Kernel::Instance().GetUser();
    PageTable* pUserPageTable = Machine::Instance().GetUserPageTableArray();

    // 将所有用户页表项清空 (标记为不存在)
    for (int i = 0; i < Machine::USER_PAGE_TABLE_CNT; i++) {
        for (int j = 0; j < PageTable::ENTRY_CNT_PER_PAGETABLE; j++) {
            pUserPageTable[i].m_Entrys[j].m_Present = 0;
        }
    }
}
```

```

// 获取物理基地址（页框号）
unsigned int textPF = u.u_procp->p_textp->x_caddr >> 12;
unsigned int pAddrPF = u.u_procp->p_addr >> 12;

// 2. 映射代码段
int numTextPages = u.u_MemoryDescriptor.m_TextSize / PAGE_SIZE;
int textStartVPN = u.u_MemoryDescriptor.m_TextStartAddress / PAGE_SIZE;

for (int i = 0; i < numTextPages; i++) {
    int vpn = textStartVPN + i;
    int pde_idx = vpn / 1024; // 页目录索引
    int pte_idx = vpn % 1024; // 页表索引

    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_Present = 1;
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_ReadWriter = 0; // 只读
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_User = 1;
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_PageBaseAddress = textPF + i;
}

// 3. 映射数据段
int numDataPages = u.u_MemoryDescriptor.m_DataSize / PAGE_SIZE;
int dataStartVPN = u.u_MemoryDescriptor.m_DataStartAddress / PAGE_SIZE;
// 物理上，数据段在 PPDA(1 页)之后
int dataPhysicalOffset = 1;

for (int i = 0; i < numDataPages; i++) {
    int vpn = dataStartVPN + i;
    int pde_idx = vpn / 1024;
    int pte_idx = vpn % 1024;

    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_Present = 1;
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_ReadWriter = 1; // 读写
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_User = 1;
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_PageBaseAddress = pAddrPF +
dataPhysicalOffset + i;
}

// 4. 映射用户栈
int numStackPages = u.u_MemoryDescriptor.m_StackSize / PAGE_SIZE;
// 栈从高地址向低地址增长，我们从栈的最高虚拟地址开始映射
int stackTopVPN = (USER_VIRTUAL_TOP_ADDRESS / PAGE_SIZE) - 1;
// 物理上，堆栈在 PPDA(1 页)和数据段之后
int stackPhysicalOffset = dataPhysicalOffset + numDataPages;

```

```

for (int i = 0; i < numStackPages; i++) {
    // 栈的虚拟地址是从高到低分配的
    int vpn = stackTopVPN - i;
    int pde_idx = vpn / 1024;
    int pte_idx = vpn % 1024;

    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_Present = 1;
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_ReadWriter = 1; // 读写
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_User = 1;
    pUserPageTable[pde_idx].m_Entrys[pte_idx].m_PageBaseAddress = pAddrPF +
stackPhysicalOffset + i;
}

// 5. 收尾阶段
FlushPageDirectory();
}

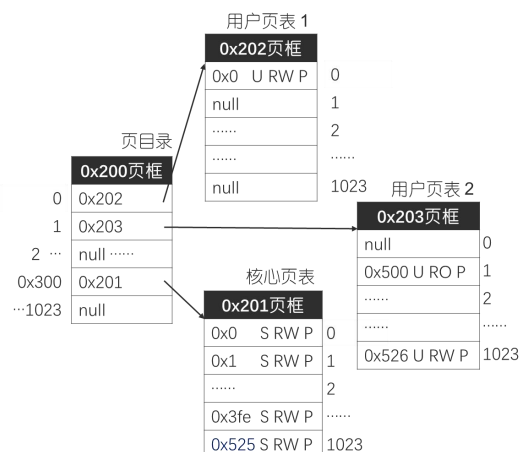
```

二、地址映射 & 内存保护

例 1: 现运行进程, 1 页代码, 1 页堆栈。
 $x_caddr = 0x500000$, $p_addr = 0x525000$ 。

(1) 填空

- 代码段第 1 条指令。
 $VA = (0x401000)$, $PA = (0x500000)$
 - user 结构
 $VA = (0xC03ff000)$, $PA = (0x525000)$
- 注: VA 虚地址, PA 物理地址



(2) 请简述 i386 MMU 地址映射、内存保护过程

- CPU 用户态执行, 读操作, $VA=0x401000$ 。

解: 地址映射、内存保护, 过程如下:

第 1 步: 查二级页表

VA 展开成 2 进制数: 00000000010000000001000000000000, 分割
 $pagetable == 1$, 查 cr3 指向的页目录, 得 $PDE[1].base = 0x203$, 映射需要的页表在 0x203#
物理页框;

$page == 1$, 查页表, 获得映射需要的 PTE, 用户页表 2 中的 $PTE[1]$ 。

第 2 步: 内存保护, 使用 $PTE[1]$ 中的保护位 prot

P 为 1, 目标逻辑页存在

CPU 用户态运行 & U, 合法

内存读操作 & RO, 合法

第 3 步: 地址映射, 使用 $PTE[1]$ 中的物理页框号 base

生成物理地址, 0x500000。

- `mov %ebp, 0xC03ff000` (CPU 核心态运行时执行这条指令)
`mov` 指令写内存单元，现运行进程的 `user` 结构的第 1 个字段。访问操作数时，CPU 使用的 `VA=0xC03ff000`。

查二级页表，确定映射需要的 PTE：核心页表、最后 1 个表项，红框所示。

通过内存保护检测，生成物理地址 `0x525000`。

- `mov %ebp, 0xC03ff000` (CPU 用户态运行时执行这条指令)
CPU 用户态运行，访问核心页面（目标 PTE 有 S 标识）。非法，MMU 抛出 14#缺页异常，杀死现运行进程，阻止其破坏系统。

- 写 `VA=0x401000` (CPU 用户态运行时执行这条指令)
目标 PTE 的属性是 RO，只读。写只读逻辑页，非法，MMU 抛出 14#缺页异常，杀死现运行进程，阻止其破坏系统。

- 读 `VA=0xC01a8000` (CPU 用户态运行时执行这条指令)
目标 PTE 的属性是 S。应用程序访问核心逻辑页，非法，MMU 抛出 14#缺页异常，杀死现运行进程，阻止其破坏系统。

- 读 `VA=0x480000` (CPU 用户态运行时执行这条指令)
映射需要的 PTE 不存在，非法~~~

- 读 `VA=0x900000` (CPU 核心态运行时执行这条指令)
映射需要的 PTE 不存在，非法~~~

[旁白，指令的执行过程，`call main1` 为例]

`call` 指令读内存单元，取 `main1` 函数第 1 条指令，使用的 `VA=0x401000`。经地址映射后，得 `PA=0x500000`。将其放地址总线，1 个主存周期后，数据总线上会出现 `main1` 函数的第 1 条指令，CPU 执行单元读数据总线获得该指令。译码、执行这条指令。

习题：PA 进程代码段 3 页，数据段 1 页，堆栈段 1 页；`x_caddr=0x402000`，`p_addr=0x407000`。

(1) 填空

- 代码段第 1 条指令。

`VA = (0x401000)`，`PA = (0x402000)`

- `user` 结构

`VA = (0xC03ff000)`，`PA = (0x407000)`

(2) 请简述下述指令执行时，i386 MMU 地址映射、内存保护过程

- `call main1` (CPU 用户态运行时执行这条指令)

第 1 步：查二级页表

VA 展开成 2 进制数：0000000001 0000000001 000000000000，分割

pagetable == 1, 查 cr3 指向的页目录，得 PDE[1].base = 0x203, 映射需要的页表在 0x203# 物理页框；

page == 1, 查页表, 获得映射需要的 PTE, 用户页表 2 中的 PTE[1]。

第 2 步: 内存保护, 使用 PTE[1]中的保护位 prot

P 为 1, 目标逻辑页存在

CPU 用户态运行 & U, 合法

内存读操作 & RO, 合法

第 3 步: 地址映射, 使用 PTE[1]中的物理页框号 base

生成物理地址, 0x402000。

- `mov %ebp, 0xC03ff000` (CPU 核心态运行时执行这条指令)

`mov` 指令写内存单元, 现运行进程的 `user` 结构的第 1 个字段。访问操作数时, CPU 使用的 `VA=0xC03ff000`。

MMU 将 `VA 0xC03ff000` 翻译为: 页目录索引 768 -> 核心页表 (0x201) -> 页表索引 1023 -> 物理页框号 0x407。

内存保护检查:

存在位: PTE 的 `P=1`, 存在。通过。

权限级别: CPU 处于核心态。PTE 标记为 `S (Supervisor)` 页面, 核心态访问核心态页面是允许的。通过。

读写权限: 操作为写操作。PTE 标记为 `RW (可读写)`。通过。

所有检查通过。MMU 成功将虚拟地址 `0xC03ff000` 映射到物理地址 `0x407000`, 并允许写入。操作合法。

- `mov %ebp, 0xC03ff000` (CPU 用户态运行时执行这条指令)

`mov` 指令写内存单元, 现运行进程的 `user` 结构的第 1 个字段。访问操作数时, CPU 使用的 `VA=0xC03ff000`。

MMU 将 `VA 0xC03ff000` 翻译为: 页目录索引 768 -> 核心页表 (0x201) -> 页表索引 1023 -> 物理页框号 0x407。

内存保护检查:

存在位: `P=1`。通过。

权限级别: CPU 处于用户态。PTE 标记为 `S (Supervisor)` 页面。用户态程序试图访问一个只允许核心态访问的页面。权限不足, 检查失败!

非法, MMU 抛出 14#缺页异常, 杀死现运行进程, 阻止其破坏系统。

- 写 `VA=0x401000` (CPU 用户态运行时执行这条指令)

MMU 将 `VA 0x401000` 翻译为: 页目录索引 1 -> 用户页表 2 (0x203) -> 页表索引 1 -> 物理页框号 0x402。

内存保护检查:

存在位: `P=1`。通过。

权限级别: CPU 处于用户态, PTE 标记为 `U (User)`。通过。

读写权限: 操作为写操作。但 PTE 标记为 `RO (Read-Only, 只读)`。程序试图写入一个只读的内存页 (代码段)。权限不足, 检查失败!

非法, MMU 抛出 14#缺页异常, 杀死现运行进程, 阻止其破坏系统。

- 读 `VA=0xC01a8000` (CPU 用户态运行时执行这条指令)

地址翻译: VA 0xC01a8000 的页目录索引是 768 (0x300)，这指向了核心页表。

内存保护检查:

权限级别: CPU 处于用户态。该地址属于内核空间 (> 0xC0000000)，因此它对应的 PTE 一定是标记为 S 的。用户态程序试图读取核心态页面。权限不足，检查失败！

非法，MMU 抛出 14#缺页异常，杀死现运行进程，阻止其破坏系统。

- 读 VA=0x480000 (CPU 用户态运行时执行这条指令)

映射需要的 PTE 不存在，非法

- 读 VA=0x900000 (CPU 核心态运行时执行这条指令)

映射需要的 PTE 不存在，非法